

Vidhooor Legal Copilot - Project Features & Tools Analysis

Project Overview

Vidhooor Legal Copilot is an Indian legal assistant application featuring:

- FastAPI backend for legal Q&A, retrieval, OCR/FIR analysis, drafting, and history
- React + Vite frontend for chat, evidence workflows, and draft operations
- ChromaDB for retrieval indexing and Oracle for persistent user/session data

Backend Features

Core Technology Stack

- **Framework:** FastAPI + Uvicorn
- **Vector Database:** ChromaDB (vector + hybrid retrieval helpers)
- **Primary Database:** Oracle DB (oracledb) for sessions/messages/evidence/drafts
- **LLM Provider:** Cerebras LLM via langchain-cerebras
- **PII Protection:** Presidio for masking/unmasking sensitive data
- **Web Scraping:** BeautifulSoup + requests for live Indian Kanoon link extraction

Key Components

1. **main.py** - API entrypoint and core request orchestration
2. **chroma_manager.py** - Retrieval/indexing and metadata-aware citation flow
3. **database.py** - Oracle repositories + schema initialization
4. **llm_engine.py** - Legal/general generation pipelines and prompting
5. **pii_vault.py** - PII masking/unmasking utilities
6. **Services Directory:**
 - `services/indian_kanoon_live.py` - Live case-link extraction + relevance scoring
 - `services/ocr_vision.py` - OCR workflow utilities
 - `services/translate_helsinki.py` - Translation pipeline helpers
 - `services/draft_exporter.py` - PDF/DOCX export helpers
 - `services/draft_mailer.py` - SMTP delivery helpers

Key API Endpoints

- **Health:** GET /

- **Chat:** POST /api/chat (primary endpoint with legal/grounded responses)
- **Drafts:**
 - POST /api/drafts/generate - Generate legal draft
 - GET /api/drafts - List drafts
 - GET /api/drafts/{draft_id} - Get specific draft
 - GET /api/drafts/{draft_id}/export - Export PDF/DOCX
 - POST /api/drafts/{draft_id}/email - Email draft
- **FIR Analysis:** POST /api/fir/analyze (OCR + translation + legal analysis)
- **Evidence:**
 - GET /api/evidence - List evidence
 - GET /api/evidence/{evidence_id} - Get encrypted evidence payload
- **History:**
 - GET /api/history/sessions - List chat sessions
 - GET /api/history/sessions/{session_id} - Get session messages
 - DELETE /api/history/sessions/{session_id} - Delete session
- **Sharing:**
 - POST /api/history/sessions/{session_id}/share - Create share link
 - GET /api/shared/{share_id} - Access shared session

Environment Variables

Required:

- CEREBRAS_API_KEY - Cerebras LLM API key
- CHROMA_HOST (default: 127.0.0.1) - ChromaDB host
- CHROMA_PORT (default: 8000) - ChromaDB port
- Oracle credentials: ORACLE_USER, ORACLE_PASSWORD, ORACLE_DSN, etc.

Optional Features:

- Live Kanoon links: ENABLE_INDIAN_KANOON_LINKS, INDIAN_KANOON_MAX_LINKS, INDIAN_KANOON_TRIGGER_MODE
- OCR: OCR_SPACE_API_KEY, OCR_SPACE_ENDPOINT, etc.
- Draft email: SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, etc.

Data & Ingestion

- **Legal Resources:** Stored in backend/data/ (statutes, constitution, case law)

- **Ingestion Scripts:**
 - `ingest_legal_resources.py` - General legal resource ingestion
 - `ingest_constitution.py` - Constitution-focused ingestion
- **Chroma Persistence:** `backend/chroma/` directory for local ChromaDB storage

Frontend Features

Technology Stack

- **Framework:** Vite + React 18 + TypeScript
- **Styling:** Tailwind CSS + shadcn/ui + Radix UI
- **Content Rendering:** React Markdown (`remark-gfm`) for assistant responses
- **Authentication:** Firebase auth integration
- **Testing:** Vitest + Testing Library

Folder Structure

- **Entry Points:**
 - `src/main.tsx` - App bootstrap
 - `src/App.tsx` - Top-level app wrapper
- **Pages:**
 - `src/pages/Index.tsx` - Main chat application shell
 - `src/pages/NotFound.tsx` - Fallback route
- **Components:**
 - `src/components/ChatArea.tsx` - Message rendering, markdown, source modal
 - `src/components/ChatInput.tsx` - Input, uploads, draft action trigger
 - `src/components/VidhoorSidebar.tsx` - Session navigation
 - `src/components/LoginModal.tsx` - Auth modal
 - `src/components/NavLink.tsx` - Sidebar/navigation helper
 - `src/components/ui/` - Shared shadcn/Radix UI primitives
- **Hooks:**
 - `src/hooks/useAuth.tsx`, `useTheme.tsx`, `use-mobile.tsx`, `use-toast.ts`
- **Libraries:**
 - `src/lib/firebaseConfig.ts` - Firebase initialization
 - `src/lib/evidenceCrypto.ts` - Client-side encryption/decryption helpers

- `src/lib/utls.ts` - Utility helpers
- **Types:** `src/types/chat.ts` - Chat/citation type definitions

Key Features

- **Chat Interface:** Real-time messaging with markdown support and source citations
- **Evidence Workflows:** Secure upload, OCR processing, translation, and encrypted storage
- **Draft Operations:** Generation, viewing, editing, and export (PDF/DOCX)
- **Session Management:** Chat history, pinning, renaming, and sharing capabilities
- **Authentication:** Firebase-based user authentication with guest mode support
- **Responsive Design:** Mobile-friendly interface using Tailwind CSS and shadcn/ui

Build & Development

- **Commands:**
 - `npm install` - Install dependencies
 - `npm run dev` - Start development server (<http://127.0.0.1:5173>)
 - `npm run build` - Production build
 - `npm run test` - Run tests
 - `npm run lint` - Run linter
- **Configuration:**
 - `vite.config.ts` - Vite configuration
 - `tailwind.config.ts` - Tailwind CSS configuration
 - `tsconfig.json` - TypeScript configuration
 - **environment variables:** `VITE_API_BASE_URL` (optional, defaults to <http://127.0.0.1:8001>)

Deployment & DevOps

Backend Deployment (Oracle Cloud + Docker)

Located in `backend/deploy/`:

- `deploy/build_and_push_ocir.sh` - Build Docker image and push to OCIR
- `deploy/deploy_backend_oci.sh` - Pull image and start stack on OCI host
- `deploy/docker-compose.oci.yml` - Backend + ChromaDB + nginx runtime stack
- `deploy/nginx.conf` - Reverse proxy configuration
- `deploy/DEPLOYMENT.md` - Step-by-step deployment runbook

Local Development Setup

1. Backend:

```
cd backend
docker compose -f docker-compose.chroma.yml up -d # Start ChromaDB
python -m uvicorn main:app --host 127.0.0.1 --port 8001 --reload
```

2. Frontend:

```
cd frontend
npm install
npm run dev # Runs on http://127.0.0.1:5173
```

Configuration Files

- **Backend:** `.env.example` (template), `requirements.txt` (Python deps)
- **Frontend:** `package.json`, `package-lock.json` (Node.js deps)
- **Shared:** `.gitignore`, `.vscode/` (editor settings)
- **Kilo Configuration:** `.kilo/` directory for AI tool configuration

Additional Tools & Resources

Documentation

- `README.md` - Project overview and quick start
- `backend/README.md` - Detailed backend documentation
- `frontend/README.md` - Detailed frontend documentation
- `backend/deploy/DEPLOYMENT.md` - Deployment guide

Sample Data

- `FIR.pdf` - Sample FIR document for testing OCR functionality

Utility Scripts

- Various ingestion and processing scripts in backend root
- Docker Compose files for different environments
- Shell scripts for deployment automation

Security & Privacy Features

- **PII Masking:** Automatic detection and masking of personal information using Presidio
- **Client-Side Encryption:** Evidence files encrypted client-side before transmission
- **Secure Storage:** Oracle database for persistent storage with proper credential management
- **Environment Isolation:** Secrets managed through environment variables, not hardcoded

- **CORS Protection:** Configured origins for frontend-backend communication
- **Input Validation:** Pydantic models for strict data validation on all API endpoints

Summary

Vidhoor Legal Copilot combines modern web technologies with specialized legal AI capabilities to provide a comprehensive legal assistant platform. The architecture separates concerns between a robust FastAPI backend handling legal reasoning, data persistence, and document processing, and a responsive React frontend providing an intuitive user interface for legal consultations, evidence management, and document drafting.